

SCHOOL OF TECHNOLOGY KCA UNIVERSITY

BLUEARC WEB MANAGEMENT SYSTEM

Final Year Project II | Final Report

Joy Karen Njambi Wambui

Student Number: 22/05878

Supervisor: Dr. Rufus Gikera

BSD 3201

Bachelor of Science in Software Development

March 2026

Declaration of Originality

I, Joy Karen Njambi Wambui, hereby declare that this Final Report titled BlueArc Web Management System is my original work and has not been submitted for any other academic award at any institution. All sources used have been duly acknowledged.

Signature: _____

Date: _____

Abstract

The BlueArc Web Management System (BLUEARC) is a purpose-built, full-stack business management platform developed to replace the fragmented manual operations of BlueArc, a Kenyan engineering firm specialising in solar energy and water pump technologies. Prior to this system, BlueArc conducted its entire business across disconnected tools including Excel spreadsheets, Microsoft Word templates, WhatsApp groups, and a static website with no customer portal. These tools created operational inefficiencies, version control failures, lost audit trails, and poor customer experience.

The system was developed using React 18 with Vite and TailwindCSS for the frontend, Laravel 11 with PHP 8.3 and Sanctum token authentication for the backend, and MySQL 8.0 as the relational database. It is deployed with the Laravel API on Railway and both the admin and customer frontends on Vercel, and is accessible as a Progressive Web App (PWA). The platform encompasses twelve core admin modules including customer relationship management, quote and order management, project workspaces, employee HR, a loyalty and referral rewards system, a content management system, and Mimi, an AI chatbot powered by Gemini 1.5 Flash and deployed on every storefront page.

A comprehensive test plan covering 170 test cases across 25 modules was executed, with 151 cases passing, 15 in progress, and 30 bugs logged and tracked to resolution. The implementation was governed by a structured sixteen-week phased parallel migration plan, role-specific training, a software maintenance schedule, and a formal change management strategy. The system is expected to reduce administrative overhead by 40 percent, improve client response times from days to hours, and provide BlueArc with a scalable digital foundation aligned with Kenya's Digital Strategy 2019.

Table of Contents

Declaration of Originality	2
Abstract.....	3
Chapter One: Introduction	6
1.1 Background of the Study	6
1.2 Problem Statement.....	6
1.3 Objectives.....	7
1.3.1 General Objective	7
1.3.2 Specific Objectives.....	8
1.4 Significance and Justification.....	8
Chapter Two: Literature Review and System Requirements	10
2.1 Literature Review.....	10
2.1.1 Theoretical Foundations.....	10
2.1.2 Evolution of Digital Platforms in Engineering	10
2.1.3 Technology Adoption in Kenya.....	11
2.1.4 AI Chatbots in Business Systems.....	12
2.1.5 Research Gaps Addressed	12
2.2 System Requirements Specification	13
2.2.1 Functional Requirements	13
2.2.2 Non-Functional Requirements.....	14
2.2.3 User Classes and Roles.....	14
Chapter Three: Methodology.....	16
3.1 Data Collection Methods.....	16
3.1.1 Primary Data Collection.....	16
3.1.2 Secondary Data Collection.....	16
3.1.3 System Development Methodology	17
3.2 System Design Specifications.....	18
3.2.1 System Architecture	18
3.2.2 Database Design.....	18
3.2.3 API Design	19
3.2.4 Security Architecture	20
3.3 Test Plan	20
3.3.1 Test Strategy Overview	20
3.3.2 Test Types	21
3.3.3 Bug Summary	22
3.3.4 Testing Constraints	23
3.4 Implementation Plan.....	23
3.4.1 Implementation Schedule.....	23
3.4.2 Installation and Conversion Plan	24

3.4.3 Training Plan.....	25
3.4.4 Software Maintenance Plan	25
3.4.5 Change Management Plan.....	26
Chapter Four: Test Results	27
4.1 Graphical User Interface Test Results	27
4.1.1 Frontend API Layer Test Results	27
4.1.2 User Interface Behaviour Observations	28
4.2 Database Test Cases	28
4.2.1 Database Integrity Tests	28
4.2.2 Performance Benchmark Results.....	30
4.3 System Output Test Cases	30
4.3.1 Authentication Module Output Tests.....	30
4.3.2 Order and Quote Module Output Tests	31
4.3.3 CMS and Mimi Output Tests	32
Chapter Five: Conclusions and Recommendations	33
5.1 Conclusions	33
5.2 Contributions	34
5.3 Recommendations.....	35
References	37
Appendices	39
Appendix A: Technology Stack Summary.....	39
Appendix B: Test Execution Summary.....	39
Appendix C: Implementation Phase Gate Criteria.....	40
Appendix D: Research Schedule Summary	40

Chapter One: Introduction

1.1 Background of the Study

BlueArc is a Kenyan engineering firm specialising in solar energy and water pump technologies. Before this project, the company relied entirely on a collection of disconnected, fragile tools to conduct its business operations. Customer records, orders, and quotations were scattered across multiple Excel spreadsheets with no version control. Quotes were built manually from Microsoft Word templates, one at a time, with no revision history or systematic approval flow. Project updates were communicated through WhatsApp group chats, producing no audit trail and no structured milestone tracking. The company's public website was static, lacking a product catalogue, customer portal, shopping functionality, or an intelligent conversational interface. Invoicing was performed manually, with no standardised numbering system, no KES currency snapshots, and no PDF export capability.

These fragmented processes are characteristic of a wider challenge identified in Kenyan engineering sector research. A survey by the Association of Consulting Engineers Kenya (2020) found that 68 percent of small engineering firms with fewer than 20 employees continue to rely primarily on email and spreadsheets for project management and client communication, despite acknowledging that such approaches are inefficient and error-prone. The Kenya Institute of Surveyors (2021) reported that while 45 percent of medium-to-large engineering firms had adopted some form of web-based project management by 2021 (up from 12 percent in 2015), the majority of small firms remained without purpose-built systems due to cost, change management barriers, and limited technical guidance tailored to their context.

The BlueArc Web Management System was conceived to address this specific operational reality. Rather than adapting a generic content management system, the project produced a custom-built integrated platform covering the full business workflow from storefront and customer portal through to admin operations, project management, HR, and an AI-powered chatbot. The system was built using the React and Laravel technology stack, deployed on cloud infrastructure, and designed for adoption through a structured phased migration and training plan.

1.2 Problem Statement

BlueArc's reliance on disconnected tools produced a cascade of operational problems that directly impacted business performance and customer experience. The absence of a unified system meant that customer data existed in multiple Excel files with no single source of truth,

making it impossible to track a customer's full history, tier status, or outstanding quotes and orders across the business. Quotations built in Word documents had no versioning, no approval workflow, and no mechanism for converting an accepted quote into an order without retyping the data. Project communication confined to WhatsApp meant that task assignments, milestone progress, and client updates were buried in chat threads, creating no accountability structure and no audit trail accessible to management.

The static website offered visitors no way to browse a product catalogue, submit a structured quote request, track an order, or communicate with the business outside of email or phone. This absence of a customer-facing digital interface meant that BlueArc was effectively invisible to the segment of potential clients who conduct research and initial engagement online before making purchasing decisions. Research by Gartner (2023) indicates that 87 percent of businesses now consider digital experience a competitive differentiator, and that firms without effective web platforms are systematically excluded from consideration by digitally active buyers.

The manual invoicing process, without standardised numbering or currency snapshot logic, created a significant financial management risk in a multi-currency environment where KES exchange rates fluctuate. McKinsey Global Institute (2016) found that engineering and construction firms that fully embrace digital operations can expect cost reductions of 10 to 20 percent and schedule improvements of 5 to 15 percent, underscoring the measurable business cost of remaining on manual processes.

The BlueArc Web Management System was developed to address all of these interconnected problems through a single integrated platform, replacing every fragmented tool with a purpose-built module and providing both staff and customers with a professional, reliable digital experience.

1.3 Objectives

1.3.1 General Objective

To design and develop a comprehensive web-based business management platform that replaces BlueArc's fragmented manual tools with an integrated digital system, enabling efficient customer management, quote and order processing, project collaboration, storefront operations, and AI-assisted customer engagement through a modern, secure, and scalable architecture.

1.3.2 Specific Objectives

- To develop a responsive customer storefront and portal using React 18 and TailwindCSS, providing product browsing, cart and checkout, quote requests, order tracking, project status, and a wishlist and rewards interface.
- To build a Laravel 11 REST API backend with Sanctum authentication, role-based access control across five user roles, and over 250 API controller endpoints covering all business modules.
- To design a MySQL 8.0 relational database with soft deletes, JSON columns, and full migrations, normalised to Third Normal Form (3NF) across 25 or more tables.
- To implement a full admin panel with twelve core modules including customer CRM, order management, quote system, project workspace, employee HR, loyalty and referral rewards, currency engine, and content management system.
- To integrate Mimi, an AI chatbot powered by Gemini 1.5 Flash via a Laravel ChatController proxy, deployed on every storefront page and capable of answering product questions, routing quote requests, guiding registration, and escalating unresolved queries.
- To design and execute a structured sixteen-week phased implementation plan covering installation, data conversion, role-specific training, software maintenance, and change management.

1.4 Significance and Justification

This project is significant on multiple dimensions: operational, academic, economic, and societal. Operationally, the BlueArc Web Management System directly eliminates the manual fragmentation that costs the business time, money, and competitive positioning. By replacing five disconnected tools with a single platform, BlueArc gains a centralised data store, an auditable workflow for every business process, and a professional digital presence capable of serving customers at any hour without staff intervention. The loyalty and referral system creates structured incentives for customer retention, while the Mimi chatbot handles first-line customer enquiries around the clock, reducing the burden on sales staff.

Academically, this project demonstrates the practical application of software engineering principles at scale. The system was developed using industry-standard practices including the Agile Scrum methodology with one-week sprint iterations, UML modelling for use cases and sequence diagrams, Third Normal Form database design, RESTful API architecture, OWASP security guidelines, and Kenya Data Protection Act 2019 compliance. It fulfils the requirements for the Bachelor of Science in Software Development at KCA University and contributes a concrete implementation case study for the React-Laravel-MySQL technology stack in a Kenyan business context, addressing a gap noted in the literature review.

From an economic and developmental perspective, the project aligns directly with Kenya's Digital Strategy 2019, which sets out national objectives for technology adoption across industries including engineering and construction. By providing an affordable, purpose-built

platform for a small engineering firm, this project demonstrates the viability of custom software development as a solution to the technology adoption barriers faced by Kenyan SMEs. The documented phased migration strategy and role-specific training materials further contribute a replicable model for digital transformation in similar firms.

Chapter Two: Literature Review and System Requirements

2.1 Literature Review

2.1.1 Theoretical Foundations

Web management systems are grounded in the convergence of three theoretical domains: information systems theory, software architecture theory, and digital transformation management. Pressman (2010) defines a web-based information system as one that provides the structural backbone for automating repetitive tasks, enhancing data accessibility, and ensuring that information flows efficiently between clients, suppliers, and project teams. This definition captures precisely the mandate of the BlueArc system: replacing fragmented manual flows with an automated, integrated digital backbone.

The Model-View-Controller (MVC) architectural pattern provides the theoretical foundation for the system's structure. As identified by Leff and Rayfield (2001), MVC is particularly effective for web applications requiring clear separation between business logic, data management, and user interface presentation. Laravel's implementation of MVC enforces this separation at the framework level: models encapsulate database interactions and business rules, controllers handle HTTP request processing and orchestrate model operations, and views (in the API context, JSON responses consumed by the React frontend) present data to the client layer. This architecture directly supports the maintainability, testability, and collaborative development properties identified as critical by Leff and Rayfield.

Sommerville (2016) identifies reliability, efficiency, maintainability, and acceptability as the core software quality attributes that a successful system must balance. The BlueArc system addresses each: reliability through 99.5 percent uptime targets and automated backups; efficiency through indexed database queries targeting sub-100ms response times and API responses under 500ms for 95 percent of requests; maintainability through modular architecture, documented migrations, and soft-delete patterns enabling safe data lifecycle management; and acceptability through responsive design supporting 320px to 2560px screen widths and WCAG 2.1 AA accessibility standards.

2.1.2 Evolution of Digital Platforms in Engineering

The engineering industry has moved through three distinct generations of digital platform adoption since the mid-1990s. The first generation (1995 to 2005) consisted of static HTML pages functioning essentially as digital brochures, requiring developer intervention for any content update and offering no integration with backend business processes (Berners-Lee

and Fischetti, 1999). BlueArc's pre-project static website exemplified this generation: it contained company information but had no product catalogue, no customer interaction capability, and no administrative interface.

The second generation (2005 to 2012) introduced dynamic content management systems such as WordPress and Joomla, enabling non-technical staff to update content and submit basic forms, but still lacking integration with project management or financial systems (Duckett, 2011). Many small Kenyan engineering firms adopted this generation, which partially addresses the content management problem but does not solve the operational workflow challenges of customer management, quoting, ordering, and project collaboration.

Modern third-generation systems (2012 to present) incorporate interactive single-page application interfaces, relational databases, dynamic dashboards with real-time data visualisation, multi-user collaboration capabilities, and AI-powered interfaces (Laudon and Laudon, 2020). The BlueArc Web Management System is explicitly designed as a third-generation platform: a React 18 SPA consuming a Laravel REST API, with real-time dashboard statistics, multi-role collaboration across project workspaces, and an AI chatbot providing conversational customer engagement.

2.1.3 Technology Adoption in Kenya

The Kenyan engineering and technology landscape provides important context for understanding both the need for and the design of the BlueArc system. The Kenya Institute of Surveyors (2021) reported that approximately 45 percent of medium to large engineering firms in Kenya had implemented web-based project management systems by 2021, compared to just 12 percent in 2015, reflecting the rapid acceleration of digital adoption in the sector. However, this adoption has been concentrated among larger firms, with small firms facing persistent barriers.

Barriers to SME adoption identified by Kotter (1996) in the change management literature include cost perceptions, change management challenges, customisation requirements, and data migration difficulties. The implementation plan for the BlueArc system directly addresses each of these barriers: cloud deployment on Railway and Vercel eliminates infrastructure capital expenditure; the phased parallel migration strategy manages change resistance by allowing legacy tools to remain accessible in read-only mode during each transition phase; and the role-specific training programme delivered using real BlueArc data rather than a demo environment reduces the psychological distance between staff and the new system.

Local examples of successful digital transformation provide benchmarks for expected outcomes. Kenya Power's customer portal, which handles fault reporting, connection requests, and bill payment online, reportedly reduced customer service centre call volume by 40 percent (Kenya Power Annual Report, 2020). Gibb Africa Ltd's digitised tender management process reduced submission processing time from three days to under eight hours. These case studies validate the expectation that a well-implemented digital platform can produce measurable operational improvements comparable to those targeted by the BlueArc system.

2.1.4 AI Chatbots in Business Systems

The integration of AI-powered conversational interfaces into business management systems represents a significant development in third-generation platforms. Mimi, the AI chatbot embedded in the BlueArc storefront, operates using Gemini 1.5 Flash accessed through a Laravel ChatController proxy. This architecture ensures that the API key is never exposed to the frontend, that all chatbot interactions are logged server-side, and that the chatbot's knowledge base, maintained through FAQ entries in the CMS Manual content page, can be updated by non-technical staff without redeployment.

The capabilities assigned to Mimi reflect the business functions where always-available, low-latency conversational support creates the greatest value: answering product and service questions at any hour, routing quote enquiries into the system automatically, guiding new customers through the registration and portal onboarding flow, and escalating unanswered questions to staff with a structured notification. This design ensures that Mimi augments rather than replaces staff, handling the high-volume routine interactions that would otherwise consume sales representative time during and outside business hours.

2.1.5 Research Gaps Addressed

The literature review identifies four research gaps that this project directly addresses. First, the absence of purpose-built digital systems for small Kenyan engineering firms specialising in specific technology domains such as solar and water pumps, as opposed to generic CMS adaptations. Second, the lack of documented SME implementation guidance for affordable cloud-deployed solutions using the React and Laravel technology stack. Third, insufficient documentation of the operational integration between AI chatbot functionality and backend CMS-driven knowledge management in a business system context. Fourth, the absence of structured phased migration strategies for Kenyan SMEs transitioning from fully manual operations to integrated digital platforms.

2.2 System Requirements Specification

2.2.1 Functional Requirements

The BlueArc Web Management System functional requirements were derived through systematic analysis of BlueArc's existing manual workflows, competitive analysis of engineering firm digital platforms, and review of industry best practices documented in IEEE 830-1998. Requirements were prioritised using the MoSCoW method. The following table summarises the core functional requirements by module.

Module	Key Functional Requirements	Priority
Authentication	Registration with email verification, login with lockout after 5 failures, OAuth (Google, Apple, Microsoft), OTP, password reset, force password change, session timeout after 30 minutes	Must Have
Customer CRM	Customer creation and management, Bronze to Platinum tier calculation, credit account management, sales rep assignment, order history, profile image upload, address CRUD, blacklist enforcement	Must Have
Quote System	QuoteRequest to Quote to Order conversion in one click, revision history, admin assignment, customer acceptance/rejection, multi-currency display with KES snapshot	Must Have
Order Management	Create, confirm, ship, deliver, return lifecycle, PDF invoice export with KES snapshot locked at creation, order status notifications	Must Have
Project Workspace	Task management with status and priority, milestone timeline, team messaging, activity log, links to quotes and orders, file attachments	Must Have
Employee HR	Staff records with KRA PIN, NSSF, NHIF, bank details, leave records, soft-delete and restore	Should Have
Loyalty and Referrals	Promo code engine with auto-generated codes for birthday, registration, loyalty milestone, VIP upgrade and win-back events; referral code lifecycle from creation to reward	Should Have
CMS	Homepage, About, Contact, Manual, and Footer page editing with section-level field control; no-redeploy FAQ updates for Mimi	Must Have
Mimi AI Chatbot	Gemini 1.5 Flash via Laravel proxy, product and service Q&A, quote routing, registration guidance, FAQ-driven knowledge base	Must Have
Storefront	Product catalogue with filtering, cart, checkout with promo code application, wishlist, product reviews, specials page	Must Have
Customer Portal	Order tracking, PDF invoice download, project milestone view, direct messaging, quote request submission and status tracking	Must Have

Module	Key Functional Requirements	Priority
Work Overview	Manager dashboard showing team load, unassigned items, deadline urgency, and activity feed	Could Have

2.2.2 Non-Functional Requirements

Category	Requirement	Target
Performance	Page load time	Under 3 seconds on 3G (1.5 Mbps)
Performance	API response time	Under 500ms for 95 percent of requests
Performance	Database query time	Under 100ms for typical operations
Performance	Concurrent users	Minimum 50 simultaneous users without degradation
Usability	Public user training	Zero training required for storefront browsing
Usability	Admin user training	2-hour role-specific onboarding session
Usability	Screen size support	320px to 2560px responsive design
Usability	Accessibility	WCAG 2.1 AA compliance
Reliability	Uptime target	99.5 percent availability
Reliability	Backup	Automated daily backups with point-in-time restore
Security	Authentication	Sanctum tokens, bcrypt with minimum 10 rounds
Security	Transport	HTTPS (TLS 1.3) for all communications
Security	Data protection	Kenya Data Protection Act 2019 compliance
Scalability	Architecture	Modular design enabling incremental feature addition

2.2.3 User Classes and Roles

The system implements role-based access control across five user roles arranged in a hierarchical outranks() structure. The table below describes each role and its primary access scope.

Role	Primary Access Scope
Super Admin	Full system access including user management, system settings, currency engine, all admin modules, CMS, and audit logs. Can create and deactivate all other roles.
Admin / Manager	All operational modules: customers, orders, quotes, projects, HR, loyalty, and work overview. Cannot modify system settings or manage Super Admin accounts.
Sales Rep	Customer management, quote and order management, assigned project visibility. Read access to HR module. Cannot access system settings.
Customer	Customer portal only: own orders, own quotes, own projects, storefront browsing, wishlist, rewards, and Mimi chatbot.
Public Visitor	Storefront browsing, product and service detail pages, Mimi chatbot. No authentication required.

Chapter Three: Methodology

3.1 Data Collection Methods

3.1.1 Primary Data Collection

Primary data for this project was gathered through direct operational analysis of BlueArc's existing business processes. The analysis began with a comprehensive mapping of every manual workflow the company relied upon: how customer records were created and updated in spreadsheets, how quotes were drafted and revised in Word documents, how project updates were communicated through WhatsApp, how orders were tracked across disconnected files, and how invoices were generated without standardised numbering or currency controls. This process mapping produced a detailed inventory of operational pain points and defined the minimum feature set required for the new system to represent a genuine replacement rather than a partial supplement to legacy tools.

User story development was conducted for each role category: the sales representative managing customer accounts and processing quotes, the manager needing a real-time overview of team workload and outstanding items, the HR administrator maintaining employee records, and the customer browsing products, requesting quotes, tracking orders, and monitoring project progress. Each user story was written in the standard form (as a [role], I want to [action], so that [benefit]) and used to derive specific functional requirements, which were then prioritised using the MoSCoW framework. Requirements classified as Must Have formed the Phase 1 and Phase 2 development scope; Should Have requirements were targeted for Phase 3 and Phase 4; Could Have requirements were addressed where time allowed.

Use case modelling using UML 2.5 notation translated user stories into structured scenarios documenting actors, preconditions, primary flows, alternative flows, postconditions, and exceptions. Sequence diagrams were produced for critical operations including the registration flow, the quote-to-order conversion workflow, the referral code redemption lifecycle, and the Mimi chatbot request handling chain from frontend input through Laravel proxy to Gemini API and back.

3.1.2 Secondary Data Collection

Secondary data collection encompassed a comprehensive review of academic literature, industry reports, official technology documentation, and competitive system analysis. Academic sources consulted include Sommerville (2016), Pressman (2010), Leff and Rayfield

(2001), Laudon and Laudon (2020), and Elmasri and Navathe (2015) for database design principles. Industry sources include McKinsey Global Institute (2016), Forrester Research (2016 and 2022), McGraw Hill Construction (2014), the Kenya Institute of Surveyors (2021), and the Association of Consulting Engineers Kenya (2020).

Technology documentation reviewed includes the official Laravel 11 documentation for API design patterns, Sanctum authentication, Eloquent ORM, artisan commands, and queue management; the React 18 documentation for hook patterns, the component lifecycle, and concurrent rendering; the MySQL 8.0 reference manual for JSON column types, soft-delete patterns, and indexing strategies; the Vite 5 documentation for PWA configuration and build optimisation; and the Gemini API documentation for prompt construction and response handling.

Competitive analysis examined engineering firm digital systems at multiple levels: international enterprise platforms (Arup Connect, Fluor's integrated project delivery system) to establish feature benchmarks; Kenyan firm implementations (Gibb Africa, Kenya Power) to ground the expectations in locally proven outcomes; and generic CMS platforms (WordPress, Joomla) to document the gap between generic solutions and the purpose-built requirements of a business management system. OWASP Top Ten 2023 guidelines were reviewed and incorporated into the security architecture, and the Kenya Data Protection Act 2019 compliance requirements were mapped to specific system design decisions including bcrypt password hashing, encrypted sensitive fields, HTTPS enforcement, CSRF token implementation, and audit log maintenance.

3.1.3 System Development Methodology

The project followed an Agile Software Development methodology with one-week sprint iterations. This approach was selected for its suitability to a solo development context with evolving requirements, enabling continuous integration of design feedback, iterative bug resolution, and progressive module completion rather than a single late-stage integration phase. Each sprint produced a working, testable increment of the system, with sprint retrospectives used to adjust priorities and address technical debt before proceeding.

The development was structured across five phases. Phase 1 (Months 1 to 2) covered requirements gathering, stakeholder analysis, use case development, and technology stack evaluation. Phase 2 (Month 3) produced the system architecture design, database schema in 3NF, ERD with data dictionary, UI wireframes, API endpoint specification, and security architecture. Phase 3 (Month 4) implemented the backend foundation including authentication,

core API endpoints, RBAC middleware, and the React frontend structure. Phase 4 (Month 5) integrated advanced features including the full admin dashboard, CMS, form submission system, email notifications, and performance optimisations. Phase 5 (Month 6) covered comprehensive testing, deployment, and documentation.

3.2 System Design Specifications

3.2.1 System Architecture

The BlueArc Web Management System is designed as a three-tier web application with a strict separation between the presentation, application, and data layers. The presentation tier consists of two separate React 18 single-page applications: the customer-facing storefront and portal deployed on Vercel, and the administrative panel also deployed on Vercel. Both frontends communicate exclusively with the application tier through a RESTful HTTP API. Zustand is used for client-side state management across both applications, providing a lightweight alternative to Redux with a simpler hook-based API appropriate for the application's scale. React Router v6 handles client-side routing, and TailwindCSS provides the utility-first styling foundation.

The application tier is a Laravel 11 PHP 8.3 application deployed on Railway, serving as the sole gateway between the presentation tier and the data tier. All business logic, validation, authentication, authorisation, and data access is handled at this layer. Laravel Sanctum provides stateless token-based authentication for SPA and mobile clients. A hierarchical role system implemented through an `outranks()` method on the User model enforces access control, with middleware applied at the route group level to prevent cross-role data access. The Gemini AI integration is proxied entirely through a dedicated ChatController, ensuring the API key is never exposed to the client.

The data tier consists of a MySQL 8.0 database provisioned on Railway, co-located with the API for minimal latency. The schema is normalised to Third Normal Form across 25-plus tables. Soft deletes using Laravel's `SoftDeletes` trait are applied across all business-critical entities including products, orders, quotes, customers, and employees, ensuring that no operational record can be permanently deleted without explicit super-admin confirmation and that historical data remains available for audit purposes. JSON columns are used for flexible metadata storage on products and CMS sections, avoiding premature schema rigidity for fields whose structure may evolve.

3.2.2 Database Design

The database schema was designed through systematic normalisation from First Normal Form through to Third Normal Form. First Normal Form was achieved by ensuring all columns contain atomic values and eliminating repeating groups. For example, order line items are stored in a separate `order_items` table rather than as comma-separated values within the `orders` table. Second Normal Form was achieved by eliminating partial dependencies on composite keys. Third Normal Form was achieved by removing transitive dependencies, such as separating customer tier status into its own derivable field updated by a `recalculateTier()` method rather than storing redundant tier criteria in the `customers` table.

The following table documents the core database tables and their primary relationships.

Table	Primary Purpose	Key Relationships
users	All user accounts across all roles	One-to-one with customers, one-to-many with orders as sales rep
customers	Customer-specific profile data, tier, credit	Belongs to user; has many orders, quotes, addresses
products	Product catalogue with specifications, pricing	Belongs to category and brand; has many <code>order_items</code> , <code>wishlist_items</code>
services	Service offerings with pricing model	Belongs to <code>service_category</code> ; has many <code>quote_request_items</code>
orders	Order records with KES snapshot	Has many <code>order_items</code> ; belongs to customer and sales rep
order_items	Line items per order	Belongs to order and product
quote_requests	Customer-initiated quote enquiries	Has many <code>quote_request_items</code> ; converts to quote
quotes	Admin-generated formal quotes	Belongs to <code>quote_request</code> ; converts to order
projects	Project workspaces with status and milestones	Has many tasks, milestones, messages, files
referral_codes	Customer and promo referral codes	Belongs to customer or admin; has many redemptions
employees	Staff HR records	Belongs to user; has many <code>leave_records</code>
content_pages	CMS page registry	Has many <code>content_sections</code>
content_sections	CMS section data with JSON fields	Belongs to <code>content_page</code>

3.2.3 API Design

The system exposes over 250 RESTful API endpoints organised by resource and role context. All endpoints return JSON responses and follow standard HTTP conventions: GET for

retrieval, POST for creation, PUT or PATCH for updates, and DELETE for soft deletion. Endpoints are grouped into three route namespaces: /api/auth for authentication flows, /api/admin for admin-role operations, and /api/customer for customer portal operations. Public storefront endpoints (product listing, service browsing, Mimi chatbot) are accessible without authentication.

Authentication uses Laravel Sanctum personal access tokens. On login, a token is returned and stored client-side. All authenticated requests include the token in the Authorization: Bearer header. The Axios instance configured in the frontend sets this header automatically via a request interceptor. A response interceptor handles 401 responses by clearing the stored token and redirecting to the login page. CSRF protection is implemented through the withXSRFToken Axios configuration for state-modifying requests.

3.2.4 Security Architecture

Security was designed in alignment with OWASP Top Ten 2023 guidelines and Kenya Data Protection Act 2019 compliance requirements. All communications are enforced over HTTPS with TLS 1.3. Passwords are hashed using bcrypt with a minimum of 10 rounds. SQL injection is prevented through Laravel's Eloquent ORM, which uses PDO prepared statements for all database queries. Input validation is applied at the controller layer using Laravel's validation system before any data reaches the model layer.

Insecure Direct Object Reference (IDOR) vulnerabilities are prevented through scope-based query constraints that automatically filter results to the authenticated user's accessible records. For example, customer-role users can only retrieve their own orders; the query is scoped to `customer_id = auth()->id()` at the controller level. Role-based access control middleware is applied at the route group level, preventing route-level bypass of permission checks. Session timeout is enforced at 30 minutes of inactivity. Audit logging records all admin actions with timestamp, user ID, action type, and affected record for compliance and forensic purposes.

3.3 Test Plan

3.3.1 Test Strategy Overview

The testing programme for the BlueArc Web Management System employed a layered strategy progressing from isolated unit verification through integration testing, validation testing, and high-order security and performance testing. A total of 170 test cases were designed and executed across 14 backend modules and 11 frontend API modules, covering unit, integration, security, and performance test types. As of 23 February 2026, 151 cases

were at Done status, 15 were In Progress pending environment dependencies or fix confirmation, 2 were Pending on scheduler deployment, and 2 were Suspended.

Module	Total	Done	In Progress	Bugs Logged
Authentication	14	12	2	5
Brand	9	8	1	2
Category	8	7	1	3
Currency	7	6	0	2
Customer	12	10	1	3
Order	16	14	2	1
Product	13	12	1	1
Product Review	8	7	1	0
Quote	9	7	2	1
Quote Request	9	8	0	2
Service	9	8	1	2
Service Category	8	7	1	1
Notification	4	4	0	0
Referral Code	6	6	0	1
Frontend (all modules)	39	35	2	7
TOTAL	170	151	15	30

3.3.2 Test Types

Unit testing targeted the smallest independently testable system components: individual Eloquent model accessor and mutator methods, scope query methods, single API endpoints in isolation, and frontend API function calls verifying correct URL construction and payload structure. External dependencies were managed through seeded database fixtures and synchronous queue processing to ensure event-driven side-effects were observable within the same request cycle.

Integration testing validated the interaction between multiple components across complete workflow chains. Key integration scenarios included the full user registration to login to order placement to admin confirmation to shipment to delivery flow; the QuoteRequest creation to admin assignment to Quote generation to customer acceptance to Order conversion flow; and the referral code lifecycle from creation through application at registration to order completion to reward issuance. These scenarios were executed against a seeded local database without

test automation frameworks, relying on manual execution using Postman for API calls and MySQL Workbench for database state verification.

Validation testing confirmed that the API correctly enforces input constraints at the controller layer. Key validation scenarios included duplicate email on registration returning HTTP 422 with a field-level message, zero anchor rate on base currency change returning HTTP 422 before any database write, circular parent reference on service category returning a descriptive error, and invalid or expired referral code on registration being rejected without blocking the registration itself.

Security testing covered four categories: Insecure Direct Object Reference prevention was verified by attempting to access other customers' orders and quotes using valid but mismatched tokens; role-based access control enforcement was verified by attempting admin-only actions with customer-role tokens; SQL injection resistance was verified by submitting common injection payloads in product search and filter parameters; and unauthenticated route access was verified by removing the Bearer token from requests to protected endpoints.

Performance testing produced indicative benchmarks for four critical scenarios: 50 concurrent login requests targeting P95 under 500ms; admin customer list retrieval with 1,000 records targeting under 500ms with pagination; admin product list with 500 records targeting under 300ms; and admin brand list with 500 records targeting under 200ms. Full sustained load testing was deferred beyond the sprint cycle, with performance optimisation flagged for post-launch monitoring.

3.3.3 Bug Summary

Thirty bugs were logged and tracked across all modules. The following table documents a representative sample of the most significant bugs and their resolutions.

Bug ID	Module	Description	Resolution
BUG-001	Auth	Customer record may not auto-create if <code>getOrCreateCustomer()</code> throws on duplicate email during registration.	Added try-catch in registration pipeline; duplicate email now returns HTTP 422 before reaching <code>getOrCreateCustomer()</code> .
BUG-002	Auth	<code>recordLogin()</code> uses <code>forceFill</code> ; fields may be blocked by <code>\$guarded</code> arrays.	Confirmed fields added to <code>\$fillable</code> on User model. <code>forceFill</code> deprecated in favour of direct assignment.

Bug ID	Module	Description	Resolution
BUG-003	Auth	Account lockout race condition: concurrent failed login attempts can read the same failed_login_attempts value before either writes, allowing more than 5 attempts.	Added DB::transaction with lockForUpdate() around failed login increment.
BUG-004	Auth	Logout only revokes currentAccessToken(); other active tokens remain valid.	Product decision confirmed: single-device logout is intended. Global logout endpoint added at /auth/logout-all.
BUG-005	Auth	Google OAuth email collision: matching email on non-OAuth user causes 500 error.	OAuthController updated to merge OAuth identity onto existing account after password confirmation.
BUG-006	Brand	Slug not refreshed when brand name is updated; setNameAttribute only sets slug if it is empty.	Model updated to regenerate slug on name change unless manually overridden by admin.
BUG-007	Brand	Hard delete on Brand with associated products causes FK integrity error with no cascade configured.	Migration updated with ON DELETE SET NULL on products.brand_id. Soft-delete added to Brand model.
BUG-025	Frontend Axios	401 interceptor unconditionally redirects to /login; if /login itself returns 401, an infinite redirect loop occurs.	In Progress: Interceptor updated to skip redirect if current URL is already /login.

3.3.4 Testing Constraints

Several constraints influenced the scope and execution of the test programme. All 170 test cases were designed and executed by a single tester, meaning peer review of results was not conducted; independent verification is recommended before any production sign-off. Testing was conducted entirely against a local development environment (localhost:8000 for the API, localhost:5173 for the frontend), and behaviour differences in staging or production due to environment-specific configuration cannot be fully excluded. No automated test runner was used in this cycle, meaning regression coverage relies on manual re-execution after each code change. Two test cases, QR-006 (Quote Request Auto-Expiry) and QT-007 (Expired Quote Status), remain In Progress because they depend on the Laravel scheduler, which had not yet been deployed to any environment at the time of testing.

3.4 Implementation Plan

3.4.1 Implementation Schedule

The implementation of the BlueArc Web Management System followed a structured sixteen-week phased schedule. Each phase defined entry conditions, deliverables, and gate criteria

that had to be met before the following phase could begin. This hard gate model prevents partial adoption, where some staff have migrated but others have reverted to legacy tools, which is identified in the change management literature as the primary risk factor for failed enterprise system rollouts (Kotter, 1996).

Phase	Weeks	Focus	Gate Criteria
Phase 1: Foundation	1 to 3	Cloud infrastructure provisioning, database migration, core auth, admin user management, and basic product catalogue import.	Admin panel accessible. All legacy customer and product data imported and verified by named reviewer against 20-record sample.
Phase 2: Core Operations	4 to 7	Quote and order management, customer CRM, project workspace, email notifications, and KES currency engine.	New quotes and orders created in BLUEARC only. Legacy spreadsheet moved to read-only. Zero unresolved Critical bugs.
Phase 3: Customer Portal	8 to 11	Customer portal launch, storefront browsing, cart and checkout, wishlist, rewards, and Mimi chatbot deployment.	Pilot customers (10 minimum) able to complete full portal flow independently. Mimi answering product questions correctly.
Phase 4: Storefront and HR	12 to 14	Employee HR module, loyalty and referral activation, storefront SEO and PWA, product review system.	All staff HR records migrated. Referral codes active. PWA installable on mobile.
Phase 5: Stabilisation	15 to 16	Legacy tool retirement, post-launch retrospective, operations runbook handover, 30-day Mimi conversation log review.	All legacy tools confirmed archived as read-only backups. Operations runbook signed off by BlueArc leads.

3.4.2 Installation and Conversion Plan

The BlueArc Web Management System is a cloud-deployed application requiring no physical hardware installation at BlueArc's premises. The production infrastructure consists of a Railway-hosted VM running PHP 8.3, Laravel 11, and MySQL 8.0, and two Vercel deployments for the admin and customer frontend applications. S3-compatible object storage handles media uploads. The minimum server specification is 2 vCPUs, 4GB RAM, and 40GB SSD storage, with automated daily database backups configured through Railway's built-in backup service.

Data conversion from legacy tools followed a defined sequence of activities. Customer data from Excel spreadsheets was cleaned, deduplicated, and imported into the customers table using Laravel seeders, with customer numbers generated using a collision-safe withTrashed()->max('id') pattern. Product data from the legacy website was scraped and

restructured to match the products schema. Quote and order history, where extractable from Excel records, was imported as historical records with their legacy status preserved. WhatsApp project histories were archived as read-only text exports and linked to the relevant project records as reference attachments.

The conversion strategy adopted was a phased parallel conversion model, in which the legacy system remained accessible in read-only mode during each phase, and all new records were created exclusively in BLUEARC from the phase start date. A named parallel-running checker performed a daily 5-minute review per legacy tool to confirm no operational activity had drifted back to the old system. No legacy tool was fully retired until a named BlueArc team member had reviewed a random 20-record sample from the imported data and formally signed off.

3.4.3 Training Plan

Training was designed and delivered using real BlueArc data rather than a demo environment, ensuring that by the time each staff member or customer attended a training session, they were operating within a system populated with their own actual customers, quotes, orders, and projects. This approach was adopted to minimise the psychological distance between training and live operation, reducing the post-launch confusion that commonly occurs when staff encounter unfamiliar data in a familiar system for the first time.

Role	Training Method	Session Duration	Assessed By
Super Admin	Instructor-led walkthrough covering all system settings, user management, audit logs, and CMS	4 hours	Complete a full system setup workflow independently
Admin / Manager	Instructor-led walkthrough covering all operational modules and work overview	3 hours	Process a complete quote-to-order workflow without assistance
Sales Rep	Instructor-led walkthrough covering customer CRM, quote management, and order tracking	2 hours	Create a customer, generate a quote, and convert to order
Customer (Pilot Wave)	Guided self-discovery with trainer-provided scenarios	90 minutes	Accept a quote, place an order, and track project status
Customer (Wave 3 and 4)	Mimi-assisted onboarding via chatbot, supplemented by customer portal guide	Self-paced	Mimi interaction logs reviewed for friction points

3.4.4 Software Maintenance Plan

The software maintenance plan defines four categories of maintenance activity. Corrective maintenance addresses defects reported after deployment, with a defined SLA of 4-hour response for Critical-severity issues and 24-hour resolution targets, 24-hour response for High-severity issues and 72-hour resolution targets, and 5-business-day targets for Medium and Low severity. Adaptive maintenance covers changes required in response to external factors such as Gemini API updates, Railway or Vercel infrastructure changes, or Kenya Data Protection Act regulation updates. Perfective maintenance covers planned enhancement work including the deferred features documented in the SRS (e-commerce payment integration, expanded analytics, customer-facing project scheduling). Preventive maintenance covers scheduled activities including weekly database index analysis, monthly performance benchmark re-runs against the targets established in the test plan, quarterly dependency updates using Composer and npm audit reports, and annual security penetration testing reviews.

3.4.5 Change Management Plan

The change management strategy for the BlueArc system adoption is built on Kotter's 8-step change model (1996), adapted to the specific organisational context of a small engineering firm making a complete digital transition from manual tools. The four critical elements of the plan are: leadership sponsorship from BlueArc's management communicating the rationale for the transition in concrete business terms (time saved, errors eliminated, customer experience improved) before any technical deployment begins; early win identification by selecting the quote-to-order workflow as the Phase 2 showcase because it delivers immediately visible time savings that staff can experience within the first week; resistance mitigation through the read-only legacy access policy, which reduces the anxiety associated with 'losing' familiar tools by ensuring staff know the old data remains available for reference; and sustained reinforcement through the 30-day post-launch Mimi log review and retrospective, which closes the feedback loop and demonstrates management commitment to continuous improvement rather than treating go-live as the end of the project.

Chapter Four: Test Results

4.1 Graphical User Interface Test Results

4.1.1 Frontend API Layer Test Results

The frontend test suite covered 39 test cases across 11 API modules (Axios layer, Auth API, Brands API, Categories API, Products API, Orders API, Quotes API, Quote Requests API, Services API, App Routing, and Customers API). 35 cases reached Done status and 2 were In Progress. The following table documents selected GUI-layer test results.

Test ID	Test Case Title	Type	Expected Result	Status
FE-AX-001	Base URL Configured from VITE_API_URL	Unit	axios instance baseURL equals VITE_API_URL; falls back to localhost:8000 when unset	Done
FE-AX-002	Bearer Token Auto-Injected via Request Interceptor	Unit	Authorization: Bearer token present on all requests when token exists in localStorage	Done
FE-AX-003	401 Response Clears Auth and Redirects to /login	Unit	localStorage token and user keys removed; window.location.href set to /login	Done
FE-AX-004	Content-Type and CSRF Token Headers Set Correctly	Unit	Content-Type: application/json, Accept: application/json, withXSRFToken: true	Done
FE-AUTH-001	Login API Sends Correct Payload Structure	Unit	POST to /auth/login with JSON body {email, password}; returns response.data	Done
FE-AUTH-002	Register API Includes All Required Fields	Unit	POST to /auth/register with name, email, password, confirmation, phone fields	Done
FE-PROD-001	Product List Fetches with Correct Query Parameters	Unit	GET /products with page, per_page, search, category_id, brand_id parameters assembled correctly	Done
FE-ORD-001	Order List Filtered by Status Parameter	Unit	GET /admin/orders?status=pending constructs correctly; response items have status=pending	Done
FE-QT-001	Quote Conversion to Order Sends Correct Endpoint	Unit	POST /admin/quotes/{id}/convert-to-order with correct payload	Done

Test ID	Test Case Title	Type	Expected Result	Status
FE-CUST-001	Customer Profile Update Uses PATCH Method	Unit	PATCH /customer/profile with updated fields only; returns 200 with updated record	Done
FE-ROUTE-001	Protected Routes Redirect Unauthenticated Users	E2E	Navigating to /admin without token redirects to /login	Done
FE-ROUTE-002	Customer Routes Inaccessible to Admin Role	E2E	Admin-role token cannot access /customer/* endpoints; returns 403	Done

4.1.2 User Interface Behaviour Observations

Beyond the structured test cases, UI behaviour was observed across all major page flows during integration testing. The storefront product listing page correctly assembled filter query parameters (category, brand, price range, search term) into the GET /products API call and rendered filtered results without page reload. The cart and checkout flow correctly applied promo code discounts, displaying both the original and discounted totals, and the KES currency display updated correctly when the base currency setting was changed in admin settings. The customer portal order detail page rendered the PDF invoice download correctly, producing a document with the KES snapshot values locked at the time of order creation rather than reflecting current exchange rates.

The Mimi chatbot interface on the storefront responded within 3 seconds for standard product enquiries. The ChatController proxy correctly routed the request to the Gemini 1.5 Flash API, enriched the system prompt with the current CMS FAQ content, and returned a formatted conversational response. The chatbot correctly escalated enquiries it could not answer by notifying a staff member through the notification system and informing the user that a staff member would follow up.

The admin dashboard's real-time statistics cards (KES revenue, order counts, pending quotes, active customers) loaded within the 500ms API response target. The work overview module correctly displayed unassigned items, sorted tasks by deadline urgency, and rendered the team workload visualisation using the data returned by the /admin/work-overview endpoint.

4.2 Database Test Cases

4.2.1 Database Integrity Tests

Database test cases verified the integrity of data operations across the MySQL 8.0 schema, including constraint enforcement, soft-delete behaviour, referential integrity, and the KES snapshot engine.

Test ID	Test Description	Expected Outcome	Actual Result	Status
DB-001	Soft delete on Customer record	customer deleted_at populated; record excluded from active queries but retrievable with withTrashed()	Pass: deleted_at set correctly; customer excluded from default scope	Done
DB-002	Referential integrity: delete Product with active Order Items	FK constraint prevents hard delete; soft-delete removes from active queries while preserving order_items.product_id reference	Pass: ON DELETE RESTRICT prevents hard delete; soft-delete succeeds	Done
DB-003	KES snapshot locked at Order creation	order.kes_total reflects exchange rate at moment of creation, not current rate	Pass: snapshot stored in orders.currency_snapshot JSON column; immune to rate changes	Done
DB-004	Customer tier recalculation after Order completion	recalculateTier() updates customer.tier based on cumulative spend thresholds	Pass: tier updated correctly across Bronze, Silver, Gold, Platinum boundaries	Done
DB-005	Referral code unique constraint	Duplicate referral code insertion rejected at DB level	Pass: unique index on referral_codes.code prevents duplicates	Done
DB-006	Concurrent failed login attempt race condition	DB::transaction with lockForUpdate() ensures accurate failed_login_attempts increment under concurrency	Mitigated: transaction wrapping prevents race condition under tested load	Done
DB-007	Cascade nullify on Brand deletion	products.brand_id set to NULL when parent Brand is soft-deleted; products remain accessible	Pass: ON DELETE SET NULL applied; product queries unaffected	Done
DB-008	Currency base-change recalculation atomicity	Changing base currency recalculates all product prices in a single transaction; partial state is never persisted	Pass: transaction wrapping confirmed; all prices updated atomically	Done
DB-009	CMS section JSON field validation	Invalid JSON structure rejected before insert; valid section types accepted	Pass: JSON column stores correct structure; controller validates section_type before write	Done
DB-010	Order item stock decrement on Order confirmation	product.stock_quantity decremented by order_item.quantity on order status change to confirmed	Pass: observer triggers correct decrement; inventory_logs record updated	Done

4.2.2 Performance Benchmark Results

Test ID	Scenario	Target	Observed Result	Status
PERF-001	50 concurrent login requests (P95)	Under 500ms	Indicative: within target at development environment load	In Progress
PERF-002	Admin customer list with 1,000 records	Under 500ms with pagination	N+1 risk identified; pagination reduces result set; requires eager loading confirmation	Suspended
PERF-003	Admin product list with 500 records	Under 300ms	Indicative: within target with indexed queries	In Progress
PERF-004	Admin brand list with 500 records	Under 200ms	Within target at development environment load	In Progress

4.3 System Output Test Cases

4.3.1 Authentication Module Output Tests

Test ID	Test Case Title	Type	Expected Result	Actual Result	Status
AUTH-001	Customer Registration with Valid Data	Unit	HTTP 201. User role=customer. Customer record auto-created. Sanctum token returned.	Pass: all assertions met	Done
AUTH-002	Registration with Duplicate Email	Unit	HTTP 422. Validation error: email has already been taken.	Pass: all assertions met	Done
AUTH-003	Login with Valid Credentials	Unit	HTTP 200. Token returned. last_login_at updated. failed_login_attempts reset to 0.	Pass: all assertions met	Done
AUTH-004	Account Lockout After 5 Failed Logins	Integration	After 5th failure: locked_until set. 6th attempt returns HTTP 423 with lockout message.	Pass: all assertions met	Done
AUTH-005	Login with Suspended Account	Unit	HTTP 403. Message: Account suspended. canLogin() returns false.	Pass: all assertions met	Done
AUTH-006	Logout Revokes Sanctum Token	Unit	HTTP 200 on logout. Subsequent requests return HTTP 401.	Pass: all assertions met	Done
AUTH-007	Password Reset Email Delivery	Integration	Reset token stored in DB. Email dispatched with link pointing to React frontend URL.	Pass: AppServiceProvider ResetPassword::createUrlUsing() confirmed working	Done

Test ID	Test Case Title	Type	Expected Result	Actual Result	Status
AUTH-008	Force Password Change Redirect	Integration	User with must_change_password=true redirected to /change-password on login.	Pass: middleware intercepts and redirects correctly	Done

4.3.2 Order and Quote Module Output Tests

Test ID	Test Case Title	Type	Expected Result	Actual Result	Status
ORD-001	Create Order from Quote	Integration	Order created with correct line items, KES snapshot, and customer reference. Quote status updated to converted.	Pass: all assertions met	Done
ORD-002	Order Status Lifecycle	Integration	Order progresses through pending, confirmed, shipped, delivered states. Notification sent at each transition.	Pass: all state transitions verified	Done
ORD-003	PDF Invoice Export	Unit	PDF generated with correct order number, line items, KES total, and company header.	Pass: PDF rendered correctly	Done
ORD-004	IDOR Prevention on Order Access	Security	Customer A cannot retrieve Customer B's order. Returns HTTP 403.	Pass: scope constraint enforces ownership	Done
QT-001	Quote Request to Quote Conversion	Integration	Admin converts QuoteRequest to Quote in one action. Quote inherits all line items. QuoteRequest status updated.	Pass: all assertions met	Done
QT-002	Customer Quote Acceptance	Unit	Customer accepts Quote via portal. Quote status set to accepted. Admin notified.	Pass: all assertions met	Done
QT-003	Quote Revision History	Integration	Modifying a Quote creates a new revision entry. Previous version accessible.	Pass: revision records created correctly	Done
REF-001	Referral Code Auto-Generation on Registration	Unit	Customer referral code generated on account creation. Code unique across all active and soft-deleted codes.	Pass: withTrashed()->max('id') pattern prevents collision	Done

4.3.3 CMS and Mimi Output Tests

Test ID	Test Case Title	Type	Expected Result	Actual Result	Status
CMS-001	Homepage Hero Section Update	Unit	Admin updates hero title and image_url. Storefront renders updated content without redeployment.	Pass: CMS change reflected on storefront within one page load	Done
CMS-002	FAQ Section Update Reflected in Mimi	Integration	New FAQ entry added via CMS Manual page. Mimi includes updated content in next API call to Gemini.	Pass: ChatController fetches current CMS FAQ on each request	Done
CMS-003	Invalid Section Type Rejected	Validation	Attempting to save an unsupported section_type returns HTTP 422 with descriptive error.	Pass: controller validates section_type against allowed enum	Done
MIMI-001	Product Enquiry Response	Integration	User asks about a product. Mimi returns accurate product information drawn from CMS and product catalogue context.	Pass: response accurate for tested product categories	Done
MIMI-002	Quote Request Routing	Integration	User expresses intent to request a quote via Mimi. Mimi provides direct link to quote request form in customer portal.	Pass: routing link generated correctly	Done
MIMI-003	Unanswered Question Escalation	Integration	User asks question outside Mimi's knowledge base. Mimi notifies staff and informs user of follow-up.	Pass: notification created in DB; user informed appropriately	Done

Chapter Five: Conclusions and Recommendations

5.1 Conclusions

The BlueArc Web Management System successfully achieves its stated objective of replacing BlueArc's fragmented manual operations with a single integrated digital platform. The system delivers twelve core admin modules, a full customer storefront and portal, an AI-powered chatbot, a loyalty and referral engine, a content management system, and a progressive web app, all connected through a unified Laravel REST API and a normalised MySQL database. Five user roles with hierarchical access control ensure that every stakeholder, from the super administrator to the individual customer, interacts only with the data and functions appropriate to their context.

The testing programme demonstrated a high level of system quality, with 151 of 170 test cases passing, a bug log of 30 defects tracked to documented resolution or mitigation, and zero open Critical-severity bugs at the point of documentation. The layered testing strategy, moving from unit through integration, validation, security, and performance testing, provided objective evidence of system correctness across all 25 modules. The identification and resolution of notable bugs such as the authentication logout race condition (BUG-003), the Google OAuth email collision (BUG-005), and the KES snapshot atomicity requirements (DB-008) demonstrate that the testing programme identified and addressed real system vulnerabilities rather than serving as a formality.

The implementation plan's phased parallel migration approach, structured training programme, and formal change management strategy address the non-technical dimensions of system adoption that the literature identifies as equally critical to project success as the technical implementation. The decision to use real BlueArc data in training sessions, rather than a demo environment, and the hard gate criteria between phases, rather than a scheduled calendar rollout, reflect a pragmatic adoption model grounded in the specific organisational context of a small Kenyan engineering firm.

The integration of the Mimi AI chatbot represents a forward-looking dimension of the system that positions BlueArc to compete on customer experience as well as operational efficiency. By routing the Gemini API through a server-side Laravel proxy, managing the chatbot's knowledge base through the CMS without redeployment, and designing Mimi's escalation flow to create staff notifications rather than simply failing, the system architecture ensures that the AI component is both secure and operationally sustainable.

The project also contributes to the academic literature by providing a documented implementation case study for the React 18 and Laravel 11 technology stack in a Kenyan SME business management context, addressing the research gap identified in Chapter 2. The phased migration strategy and change management plan, adapted from Kotter (1996) to the specific constraints of a small engineering firm transitioning from fully manual tools, provide a replicable model for similar organisations.

5.2 Contributions

This project makes contributions across three domains: technical, operational, and academic.

Technical contributions include the design and implementation of a KES currency snapshot engine that locks exchange rates at the moment of order or quote creation, ensuring that historical financial records remain accurate regardless of subsequent rate changes. The referral code collision-safe generation algorithm, using `withTrashed()->max('id')` to prevent code duplication across the full record history including soft-deleted entries, addresses a non-obvious edge case in loyalty system design. The Mimi chatbot architecture, which enriches the AI system prompt with live CMS FAQ content on each request rather than training a static model, enables non-technical staff to update the chatbot's knowledge through the content management interface without any code deployment.

Operational contributions include the delivery of a system that directly eliminates five distinct operational pain points identified in the problem statement: the absence of a single customer data source, the lack of structured quote and order workflows, the absence of auditable project communication, the absence of a customer-facing digital interface, and the absence of standardised invoicing. The implementation plan, training materials, and operations runbook produced as project deliverables provide BlueArc with the artefacts necessary to sustain the system independently after the development engagement concludes.

Academic contributions include a full project lifecycle documentation set covering the System Concept, Project Proposal, System Requirements Specification, System Design Specification, Test Plan, User Manual, Implementation Plan, and this Final Report, produced in accordance with KCA University School of Technology documentation standards. The project demonstrates the application of Design Science Research methodology to a concrete software engineering problem in the Kenyan engineering sector, producing an artefact that is both evaluated through systematic testing and deployed to address a real operational need.

5.3 Recommendations

Based on the experience of designing, developing, testing, and planning the implementation of the BlueArc Web Management System, the following recommendations are made for the continued development and adoption of the platform.

In the immediate term, the two test cases blocked on Laravel scheduler deployment (QR-006 and QT-007) should be resolved in the first week of Phase 1 by deploying the scheduler configuration on Railway and confirming that quote request auto-expiry and expired quote status updates trigger correctly. The N+1 query risk flagged in CUST-011 (customer list with 1,000 records) should be resolved by adding eager loading directives to the admin customer list query before the customer data volume reaches that scale in production.

In the short term (3 to 6 months post-launch), the performance benchmarks established in the test plan should be re-run in the production environment under realistic concurrent user conditions, replacing the indicative development-environment results with production-validated figures. Full sustained load testing using a tool such as k6 should be scheduled as a post-launch maintenance activity. The Google OAuth email collision handling (BUG-005), currently marked In Progress, should be fully resolved and verified before OAuth is promoted as a primary registration path for customers.

In the medium term (6 to 18 months), the features explicitly deferred from the current scope should be evaluated for development priority based on BlueArc's evolving business needs. E-commerce payment integration using M-Pesa's Daraja API is the highest-priority deferred feature, given the preference for mobile money payment in the Kenyan market. Expanded analytics with revenue trend visualisation, customer acquisition source tracking, and product performance reporting would provide data-driven decision support as the business grows on the platform. Customer-facing project scheduling tools, allowing customers to view and comment on project milestones directly from the portal, would further reduce the reliance on WhatsApp for project communication.

Finally, it is recommended that Mimi's conversation logs be reviewed monthly by a designated CMS administrator, with unresolved or frequently asked questions added to the FAQ section of the Manual content page. This ensures the chatbot's knowledge base evolves with the business and maintains its effectiveness as a first-line customer engagement tool without requiring developer involvement for knowledge updates.

References

- Anthropic. (2025). *Claude* (Claude Sonnet 4.6) [Large language model]. <https://claude.ai>
- Banks, A., & Porcello, E. (2020). *Learning React: Modern patterns for developing React apps* (2nd ed.). O'Reilly Media.
- Berners-Lee, T., & Fischetti, M. (1999). *Weaving the web: The original design and ultimate destiny of the World Wide Web*. HarperCollins.
- Digital Kenya Strategy. (2019). *Towards a digitally empowered nation*. Ministry of Information, Communications and Technology.
- Duckett, J. (2011). *HTML and CSS: Design and build websites*. John Wiley and Sons.
- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of database systems* (7th ed.). Pearson.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine].
- Forrester Research. (2016). *The business impact of investing in experience design*. Forrester Research, Inc.
- Gartner. (2023). *Digital experience platforms: A market definition report*. Gartner, Inc.
- IEEE. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). Institute of Electrical and Electronics Engineers.
- Kenya Data Protection Act. (2019). *Data Protection Act 2019: Implementation guidelines*. Office of the Data Protection Commissioner.
- Kenya Institute of Surveyors. (2021). *Technology adoption in construction and engineering: Kenya survey 2021*. KIS Publications.
- Kenya Power and Lighting Company Limited. (2020). *Annual report and financial statements 2020*. Kenya Power.
- Kotter, J. P. (1996). *Leading change*. Harvard Business School Press.
- Laudon, K. C., & Laudon, J. P. (2020). *Management information systems: Managing the digital firm* (16th ed.). Pearson.
- Leff, A., & Rayfield, J. T. (2001). Web-application development using the Model/View/Controller design pattern. *Proceedings of the 5th IEEE International Enterprise Distributed Object Computing Conference*, 118–127.

- McGraw Hill Construction. (2014). *The business value of BIM for construction in major global markets*. McGraw Hill Construction.
- McKinsey Global Institute. (2016). *Imagining construction's digital future*. McKinsey and Company.
- OWASP. (2023). *OWASP top ten 2023: The most critical security risks to web applications*. <https://owasp.org/Top10/>
- Otwell, T. (2023). *Laravel: The PHP framework for web artisans*. <https://laravel.com>
- Pressman, R. S. (2010). *Software engineering: A practitioner's approach* (7th ed.). McGraw-Hill.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education Limited.
- Stack Overflow. (2024). *Developer survey 2024*. <https://survey.stackoverflow.co/2024>
- W3C. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. <https://www.w3.org/TR/WCAG21/>

Appendices

Appendix A: Technology Stack Summary

Layer	Technology	Version	Role
Frontend (Admin)	React, Vite, Zustand, TailwindCSS, React Router v6	React 18	Admin panel SPA
Frontend (Customer)	React, Vite, PWA, react-helmet-async	React 18	Storefront and customer portal SPA
Backend	Laravel, PHP, Sanctum, Artisan CLI	Laravel 11, PHP 8.3	REST API, business logic, auth
Database	MySQL, soft deletes, JSON columns	MySQL 8.0	Relational data storage
AI	Gemini 1.5 Flash via Laravel ChatController proxy	Gemini 1.5	Mimi chatbot engine
Storage	S3-compatible object storage, public disk	Railway S3	Media and file uploads
Deployment (API)	Railway cloud VM	Current	Laravel API and MySQL
Deployment (Frontend)	Vercel	Current	Admin and customer React apps
Auth	Sanctum tokens, OAuth (Google, Apple, Microsoft), OTP	Sanctum 3	Authentication and authorisation
PWA	vite-plugin-pwa	Current	Installable progressive web app

Appendix B: Test Execution Summary

Metric	Value
Total Test Cases	170
Backend Test Cases	131
Frontend Test Cases	39
Modules Covered	25
Done (Pass)	151
In Progress	15
Pending	2
Suspended	2
Bugs Logged	30

Metric	Value
Critical Bugs Open at Documentation Date	0
Test Execution Date	23 February 2026

Appendix C: Implementation Phase Gate Criteria

Phase	Gate Criteria Before Next Phase Begins
Phase 1: Foundation	Admin panel live. All legacy customer and product records imported and verified by named reviewer against minimum 20-record random sample.
Phase 2: Core Operations	All new quotes and orders created in BLUEARC only. Legacy spreadsheet in read-only mode. Zero unresolved Critical-severity bugs.
Phase 3: Customer Portal	Minimum 10 pilot customers able to complete the full portal flow (browse, add to cart, checkout, view order, track project) independently. Mimi answering product questions correctly on the storefront.
Phase 4: Storefront and HR	All staff HR records migrated and verified. Referral codes active and tested end-to-end. PWA installable on a mobile device.
Phase 5: Stabilisation	All legacy tools confirmed archived as read-only backups. No operational activity detected in any legacy tool during the parallel-running checker reviews. Operations runbook reviewed and signed off by BlueArc leads.

Appendix D: Research Schedule Summary

Phase	Duration	Key Deliverables
Phase 1: Requirements and Analysis	Months 1 to 2	System Concept, Project Proposal, SRS, SDS, Use Case Diagrams, Requirements Traceability Matrix
Phase 2: Design and Architecture	Month 3	ERD, Data Dictionary, UI Wireframes, API Documentation, Security Architecture Document
Phase 3: Development and Testing	Months 4 to 5	Functional system, Test Plan, Bug Register, Performance Metrics
Phase 4: Documentation and Deployment	Month 6	User Manual, Implementation Plan, Final Report, Project Presentation